

FoodFlow Build Plan

What we keep from the two prototypes, what we build tonight, and the order to build it in.

DEVPOST LOCKS

Sun 11:00 SGT

Judging 12:00–14:30, walking format

THE ONE DECISION EVERYTHING ELSE DEPENDS ON

FoodFlow is the codebase. Smart Dining donates its data model.

FoodFlow boots in one command, has no database to fail on demo day, and its checkout flow already answers the Trust & Safety rubric. Smart Dining is the better-engineered system, but a Postgres + Vercel deploy that isn't live yet is a liability at 11pm, not an asset.

So: **ship FoodFlow's shell and checkout, port Smart Dining's schema thinking** into a single JSON file on the Express server, and **throw away the rest of the infrastructure**. No Postgres, no Vercel, no TanStack. One `npm start`.

● Keep as-is

from foodflow-prototype

styles.css + shell

35 KB of finished dark UI. Do not touch it, do not restyle it.

3-stage checkout modal

Review → consent → OTP → transaction preview → pay. This is the rubric answer, already built.

Trace chips

The little "search_catalog · rank_by_constraints" line under each reply. Judges love visible reasoning.

Local heuristic fallback

Demo insurance. If venue wifi dies mid-pitch the whole flow still completes.

Merchant launchpad layout

Upload zone, stepper, publish row, stats. Only the wiring behind it changes.

Express + tool-calling scaffold

The function-tool plumbing works. We add tools, not rewrite it.

● Port the idea, not the code

from smart-dining-vercel

Relational shape

merchant → store → menu_item → order → order_item. Rebuild it as nested JSON, keep the foreign-key discipline.

Price snapshots on order items

An order records what the price was. Cheap to add, reads as real engineering.

Server-side price validation

Smart Dining recalculates totals server-side. FoodFlow trusts the browser. Fix this.

First-class operational fields

wait_minutes, price_tier, is_available, dietary_tags — not free-text tags.

Order status lifecycle

confirmed → preparing → ready. One line of demo polish after payment.

● Cut

dead weight

Three duplicate catalogs

Hardcoded arrays in app.js and server.js plus state.menuProducts. Collapse to one.

Postgres / Vercel / Higgsfield

Every one of them is an unconfirmed deploy. Not tonight.

The dead async intent check

app.js line 496 returns a Promise that is compared to a string. Delete it.

Geolocation + food-court browsing

Smart Dining's best feature, and completely off-brief. The brief is conversation, not browsing.

Fake copy

"3 merchants · 18 live items" when there are 6.
Judges check.

The catalog intelligence pipeline

YOUR IDEA, SPECIFIED

This is the differentiator and it maps directly to the brief's weakest-covered requirement: *"a simple way for merchants to become ready for agents to shop."* A dry Excel goes in one end; a structured, agent-queryable catalog comes out the other. Everything else in the plan exists to serve this.

01 Merchant drops a file Any CSV or Excel. Messy headers, missing columns, mixed currency — all allowed.	02 · AI Column mapping "Dish Name" / "Item" / "Menu" all map to <i>name</i> . Model reads the header row and guesses the schema.	03 · AI Attribute extraction Per dish: flavour, texture, heaviness, cooking method, dietary, allergens, meal fit.	04 · AI Clustering Groups items into category tabs that fit <i>this</i> menu, not a fixed taxonomy.
05 · AI Copy assist Writes a customer-facing line for every blank description. Flags low-confidence rows.	06 Merchant approves Editable review table. Nothing goes live without a human click. Then Publish.		

Stages 02–05 are one or two model calls over the whole sheet, not one per row — cheaper, faster, and the model sees the menu as a whole, which is what makes the clustering good.

F1 One catalog, one source of truth

must

~45 min

Nothing else works until this does. Right now a merchant can upload a menu, publish it, ask the agent about it, and get the six hardcoded demo items back. A judge will try exactly this.

F1.1 Server owns the catalog

Move the product array into `data/catalog.json`. Delete both client-side copies.

F1.2 Catalog API

GET `/api/catalog?merchant=`, POST `/api/merchant/catalog`, POST `/api/merchant/publish`.

F1.3 Client fetches on load

`app.js` pulls the catalog instead of shipping it. Cart and compare read from the same list.

F1.4 Survives refresh

Write the JSON to disk. Currently a reload wipes the published menu, the cart and the order.

Rubric: Technical Feasibility. Without it, the merchant onboarding story is a lie the demo exposes in ten seconds.

F2

Catalog intelligence — messy sheet to agent-ready menu

must

~2.5 hrs

The centrepiece. This is what makes the pitch "we make any merchant agent-ready in 15 seconds" instead of "we built a food chatbot". F2.2 is the single highest-value thing on this page.

F2.1 Schema inference

Model maps arbitrary column headers onto our fields. Accepts .xlsx as well as .csv. Reports what it could not map instead of silently dropping it.

F2.2 Attribute extraction

Every dish gets flavour profile, texture, heaviness, cooking method, temperature, dietary flags, allergens, meal fit and occasion — plus a confidence score. Schema below.

F2.3 Auto-categorisation

Model clusters the enriched items into tabs that suit the menu. A Japanese upload yields Donburi / Ramen / Small plates / Cold — not a generic Mains / Sides / Drinks.

F2.4 Description writer

Blank or terse descriptions get a one-line customer-facing rewrite. Shown as a suggestion the merchant accepts or rejects — never auto-published.

F2.5 Review & approve table

AI-added fields visually distinct from uploaded ones, inline-editable, low-confidence rows flagged for attention. This screen is the demo money shot — film it.

F2.6 Gap report

"4 items have no price. No vegetarian option in this catalog. 3 items missing allergen data." Merchant value that has nothing to do with the chatbot.

Rubric: Innovation + Scalability. Answers "customisable for different merchant types" without writing a single merchant-specific line of code.

F3**Agent that actually reasons over the attributes**

must

~2 hrs

F2 is wasted if the agent still runs regex over item names. "Something warm and soupy, not too heavy, no pork, under \$10" is unanswerable today and trivially answerable once F2 lands.

F3.1 Attribute-scored retrieval

Replace the keyword matcher with weighted scoring across the enriched fields. Hard filters for allergens and dietary; soft scores for flavour and texture.

F3.2 Fix add_to_cart

The server already computes the new cart and then throws it away — the response never returns it and the client never reads it. The agent literally cannot add to cart right now.

F3.3 Fix compare_items

Return a real attribute diff and render it. Today the comparison array is computed, sent, and ignored.

F3.4 check_allergens tool

A direct, auditable "is there shellfish in this" answer. Cheap to build, strong Trust signal.

F3.5 One clarifying question

When a request is underspecified the agent asks exactly one question before recommending. Makes it feel like an agent, not a search box.

F3.6 Explain the pick

"Soupy, no pork, 12 min — matches what you asked for." Grounds every recommendation in a stated constraint.

Rubric: User Experience. This is the part the judge experiences directly at your station.

F4 Payment and consent hardening

should

~1 hr

The visible flow is already good. What's missing is the part that makes it defensible when a Visa engineer asks "where is the total calculated?"

F4.1 Server recalculates the total

Never trust the browser's number. Reject the payment if the client total and server total disagree.

F4.2 Agent action ledger

Record what the agent proposed, what the user approved, and when. Show it on the receipt. This is the single most Visa-ish thing we can add.

F4.3 Spend cap

The agent cannot prepare an order above a set amount without a second explicit confirmation.

F4.4 Idempotency key

One order, one charge, however many times the button is clicked.

Rubric: Trust & Safety. F4.2 is the line that gets quoted back to you in the closing ceremony.

F5

The scalability story

could

~45 min

A rubric line worth real points that costs almost nothing, because F1 already keys everything by merchant.

F5.1 Multi-merchant URLs

Catalog keyed by merchantId; /m/green-bowl opens that merchant's agent. Seed two merchants so the switch is visible on screen.

F5.2 Embed snippet

One line an SME pastes into their own site: `<script src="../../../agent.js" data-merchant="green-bowl">`. Even as a copyable string on the merchant page it lands the plug-and-play claim.

F5.3 Category pack stub

Food is one pack. Show a greyed-out Fashion and Electronics selector to prove the architecture is category-agnostic. Do not build them.

Rubric: Scalability. Cheapest points on the board — but only after F1–F3 are done.

What the model returns per dish. Keep it a fixed shape — closed vocabularies for anything the retrieval scores on, free text only for description. Adding a field costs nothing at generation time and a lot at query time, so decide this once.

```
{
  "id": "katsu-don",
  "name": "Chicken Katsu Don",
  "price": 8.50,
  "description": "Crisp fried cutlet over rice in sweet-savoury sauce", // AI-written

  // ---- flavour, 0–3 intensity. This is what "sweet sauce" becomes. ----
  "flavour": { "savory": 3, "sweet": 2, "salty": 2, "spicy": 0, "sour": 0, "umami": 0 },

  // ---- form and feel. This is what "soupy" becomes. ----
  "texture": ["crispy", "tender"], // crispy | soupy | chewy | creamy | crunchy
  "cooking": "deep-fried", // fried | grilled | steamed | raw | baked
  "temperature": "hot",
  "heaviness": 4, // 1 light → 5 very heavy
  "portion": "regular",

  // ---- hard filters. Never soft-scored, never guessed silently. ----
  "dietary": [], // vegetarian | vegan | halal | no-bee
  "contains": ["chicken", "egg", "gluten"],
  "allergens": ["egg", "gluten", "soy"],

  // ---- context for recommendation ----
  "mealFit": ["lunch", "dinner"],
  "occasion": ["comfort", "quick-solo"],
  "cuisine": "Japanese",
  "category": "Donburi", // assigned by clustering, F2.3
  "prepMinutes": 12,

  // ---- provenance. Drives the review screen. ----
  "confidence": 0.82,
  "needsReview": false,
  "aiFields": ["description", "flavour", "texture", "allergens", "category"]
}
```

Design note. `aiFields` is what lets the review table highlight exactly what the model added versus what the merchant uploaded. It costs one array and it is the difference between "the AI guessed your menu" and "the AI showed its work".

Rotate the OpenAI key

Your teammate's `.env.example` contains a live-looking `sk-proj-...` key, and `.gitignore` only excludes `.env` — so it ships in the zip and would ship in the Devpost repo. Revoke it on the OpenAI dashboard, replace the line with an empty `OPENAI_API_KEY=`, and add `.env.example` to git only after it's blank.

Decide the AI budget now

F2 makes one or two model calls per upload and F3 makes one per message. That's fine for a demo, but agree a spending cap and whose key it is before you wire it up at 2am — and keep the local heuristic fallback working so a rate limit during judging degrades the demo instead of ending it.

Eleven hours

SUGGESTED ORDER

Written for two people working in parallel. The split that matters: one person owns the server and the enrichment pipeline, the other owns the merchant review screen and the agent's conversational behaviour. They meet at F1's API contract, so agree that contract first and then don't block on each other.

TIME	WORK	SHIPS
23:45 – 00:15	Agree the API contract and the enrichment schema Thirty minutes on paper saves three hours of merge pain. Rotate the key while you're here.	—
00:15 – 01:00	Collapse the three catalogs into one server-side JSON Do this before anyone builds anything on top of it.	F1
01:00 – 03:00	Enrichment pipeline Column mapping, attribute extraction, clustering. Test it against a genuinely ugly spreadsheet, not a clean one you made.	F2.1–F2.3
01:00 – 03:00	Merchant review screen, in parallel Build against the agreed schema with fixture data while the pipeline is still being written.	F2.5
03:00 – 04:30	Attribute-based retrieval, fix the broken tools add_to_cart and compare_items are near-free wins — the server code already exists.	F3.1–F3.3
04:30 – 05:15	Payment hardening Server-side totals and the agent action ledger. Skip the rest if you're behind.	F4.1–F4.2
05:15 – 08:00	Sleep — in shifts if you have to You present at 12:00 and stand at your station for two and a half hours. Judging is not a code-quality contest and you cannot answer questions on no sleep.	—
08:00 – 09:00	Copy assist, gap report, clarifying question	F2.4, F2.6, F3.5

TIME	WORK	SHIPS
	Everything left over, in value order. Drop anything not working by 09:00.	
09:00 – 09:45	Record the demo video Dry Excel in → enriched catalog → publish → conversation → checkout → receipt. Do a silent take first, then narrate. Non-negotiable deadline.	—
09:45 – 10:30	README, architecture diagram, Devpost form The brief explicitly asks for architecture, merchant onboarding, and trust handling. Write those three sections properly — they're marked.	—
10:30 – 11:00	Buffer, then submit Submit at 10:45 with whatever is working. You can keep polishing for the 12:00 judging; you cannot resubmit at 11:01.	—

If you fall behind, cut in this order

DESCOPING

1. **F5.3 category pack stub** — a nice sentence in the pitch, worth nothing on screen.
2. **F2.6 gap report** — genuinely useful, entirely skippable for a demo.
3. **F5.2 embed snippet** — say it in the pitch instead of building it.
4. **F4.3 / F4.4 spend cap and idempotency** — mention them as "designed, not built" and you still get credit for having thought about it.
5. **F2.4 description writer** — the enrichment reads fine without rewritten copy.
6. **F3.5 clarifying question** — the conversation is flatter without it, but it still works.

Never cut: F1, F2.2, F2.5, F3.1 and the demo video. That set is the entire submission — one catalog, real attributes, a screen that shows the AI's work, an agent that uses it, and proof it runs.

Built from a read of `foodflow-prototype/` and `smart-dining-vercel/` · full code audit in the project as `claude/foodflow-prototype-audit.md`